

Tema 4 — Clases y objetos

Contenido teórico

Objetivos

- **Entender una clase** como una plantilla que define **atributos y métodos** para crear **objetos**.
- **Crear instancias** con `__init__` y usar **self** para acceder al estado propio de cada objeto.
- **Diferenciar** atributos de instancia, atributos de clase, métodos y representación textual.
- **Aplicar** encapsulación básica, composición y herencia simple en ejemplos de TI.
- **Reconocer la herencia** múltiple y sus cautelas sin convertirla en la solución por defecto.

Mapa del Tema 4

- Empezaremos con la idea de clase, objeto e instancia. Después estudiaremos atributos, métodos, constructor y representación de objetos.
 - **Clase -> define la estructura**
 - **Objeto -> instancia concreta de una clase**
 - **Atributos -> datos**
 - **Métodos -> comportamientos**
- A continuación trabajaremos la encapsulación y la composición, ya que suelen ser más útiles en código real que una jerarquía de herencia excesiva.
- Finalmente revisaremos herencia, sobrescritura de métodos, polimorfismo básico y errores frecuentes al diseñar clases.

Clase, Objeto e Instancia

Una clase define cómo serán los objetos de un tipo:

- Qué datos tendrán y qué operaciones podrán realizar.
- Es similar a crear un nuevo tipo de dato adaptado al código que estamos haciendo.

Un objeto es un elemento concreto creado a partir de una clase. A ese objeto también se le llama instancia.

- En Python todo es un objeto: cadenas, listas, enteros, funciones y también las instancias creadas por nuestras propias clases.

NOTA: Cuando estás escribiendo código, la palabra clave **pass** se usa para indicar que de momento ahí no hay nada, pero es sintácticamente correcto. Se puede usar en clases, funciones, condicionales o bucles cuando todavía no queremos escribir una implementación.

```
class Servicio:
    pass

ssh = Servicio()
web = Servicio()

print(type(ssh))
print(ssh is web)
```

Atributos y Métodos

La programación orientada a objetos agrupa información y operaciones relacionadas:

- **Los atributos:**
 - Guardan datos del objeto.
 - Se asignan como asignamos habitualmente las variables.
- **Los métodos:**
 - Definen acciones que el objeto sabe ejecutar.
 - Se declaran con `def`, como las funciones.
 - De hecho podemos pensar en un método como en una función interna al objeto.

Esto nos ayuda a organizar programas cuando ya no es suficiente con la programación tradicional (También conocida como **programación estructurada o imperativa**, consistente en variables sueltas, sentencias de control de flujo y funciones aisladas).

En un script en el sector de la TI, un servidor, un servicio, una tarea programada o una alerta pueden representarse como objetos.

```
class Servicio:
    def __init__(self, nombre, activo):
        self.nombre = nombre
        self.activo = activo

    def esta_activo(self):
        return self.activo
```

Definir una clase

Una clase se define con la palabra clave **class** y un nombre.

- Por convención, los nombres de clase usan **PascalCase**: cada palabra empieza en mayúscula.
- El cuerpo de la clase contiene métodos y, normalmente, un método inicializador llamado `__init__()`.
- **NOTA:** Los guiones bajos de inicio y fin en este método y posteriores que veremos, son dobles.
- **Definir una clase no crea todavía ningún objeto.** Solo deja preparada la plantilla para crear instancias posteriormente.

```
class EquipoRed:
    def __init__(self, nombre, ip):
        self.nombre = nombre
        self.ip = ip

# Aún no hay objetos creados
```

El método inicializador `__init__()`

El método especial `__init__()` se ejecuta automáticamente cuando se crea una instancia. Se utiliza para dejar el objeto en un estado inicial coherente.

- En otros lenguajes de POO como Java o C++ existe el concepto de constructor. En Python suele llamarse **constructor** por simplificación didáctica, aunque técnicamente `__new__()` crea el objeto y `__init__()` lo inicializa.
- Los parámetros recibidos permiten cargar datos iniciales. Dentro del método se asignan a atributos mediante `self`.

```
class EquipoRed:
    def __init__(self, nombre, ip):
        self.nombre = nombre
        self.ip = ip

router = EquipoRed("router01", "10.0.0.1")
print(router.nombre)
```

Qué significa self

self representa la instancia concreta sobre la que se está ejecutando el método. No es una palabra reservada, pero es la convención universal en Python.

- Por ejemplo, cuando llamamos **router.mostrar()**, Python pasa internamente el objeto router como primer argumento, y es el que llamamos **self**.
- Gracias a **self**, cada objeto mantiene sus propios datos aunque todos compartan la misma definición de clase.

```
class EquipoRed:
    def __init__(self, nombre):
        self.nombre = nombre

    def mostrar(self):
        print(f"Equipo: {self.nombre}")

sw = EquipoRed("switch01")
sw.mostrar()
```

Crear instancias

Crear una instancia consiste en llamar a la clase como si fuera una función.

- Cada llamada crea un objeto independiente.
- Dos objetos pueden tener los mismos valores, pero siguen siendo objetos distintos en memoria.

Esto permite modelar varios elementos reales del mismo tipo, como distintos servidores o servicios, sin duplicar la lógica.

```
class Servicio:
    def __init__(self, nombre):
        self.nombre = nombre

ssh = Servicio("ssh")
web = Servicio("httpd")

print(ssh.nombre)
print(web.nombre)
print(ssh is web)
print(ssh)
print(web)
# Muestra que son dos servicios distintos
```

Atributos de instancia

Los atributos de instancia pertenecen a cada objeto concreto. Se crean normalmente asignándolos con **self.atributo** dentro de **__init__()**.

- Cada instancia puede tener valores diferentes para esos atributos.
- Cambiar el atributo de un objeto no modifica el atributo de otro.
- Son adecuados para datos propios del objeto: nombre, IP, estado, puerto o ruta.

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

ssh = Servicio("ssh", 22)
web = Servicio("http", 80)

web.puerto = 8080
print(ssh.puerto, web.puerto)
```

Métodos de instancia

Un método de instancia es una función definida dentro de una clase que recibe `self` como primer parámetro.

- Puede leer o modificar los atributos del objeto.
- Los métodos deben representar operaciones propias del objeto.
- Si un método no usa el estado de la instancia, quizá no debería ser método de instancia.

En código real conviene que el método tenga una responsabilidad clara dentro de la clase.

```
class Servicio:
    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

    def iniciar(self):
        self.activo = True

ssh = Servicio("ssh")
print(ssh.activo)
ssh.iniciar()
print(ssh.activo)
```

Representación textual: `__str__()`

Si imprimimos un objeto sin definir una representación, Python muestra una referencia poco legible. El método especial `__str__()` permite devolver una cadena formateada y amigable.

- No debe imprimir directamente: debe devolver texto con **return**. Así el objeto puede usarse en **print()**, **f-strings** o logs.
- Es una mejora sencilla que facilita depuración y salida por pantalla.

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

# Este print devolverá salida sin sentido
print(Servicio("ssh", 22))

class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    def __str__(self):
        return f"{self.nombre}:{self.puerto}"

# Este print devolverá salida estructurada
print(Servicio("ssh", 22))
```

Atributos de clase e instancia

Un atributo de clase se define dentro de la clase, pero fuera de los métodos.

- Es compartido por todas las instancias mientras no se sobrescriba en un objeto concreto.
- Los atributos de instancia se definen con **self** y contienen datos propios de cada objeto.
- Para evitar confusión, se deben usar atributos de clase solo para valores realmente comunes o constantes del tipo.

```
class Servicio:
    categoria = "sistema"

    def __init__(self, nombre):
        self.nombre = nombre

ssh = Servicio("ssh")
web = Servicio("httpd")
print(ssh.categoria, web.categoria)
```

Encapsulación en Python

Encapsular significa controlar cómo se accede o modifica el estado interno de un objeto. Python no impone privacidad absoluta como otros lenguajes. Se apoya en convenciones.

Un atributo con un guion bajo inicial, como **_estado**, indica uso interno. Un doble guion bajo, como **__estado**, activa **name mangling** y dificulta el acceso directo del atributo. Esto no equivale a una privacidad absoluta.

La idea práctica es no modificar libremente el estado de los atributos externamente con asignaciones, sino obligar a usar métodos **get()** y **set()**.

Para evitar confusiones:

- **Un guion inicial (_atributo)**: Convención de "uso interno". Es solo un aviso visual, no cambia nada técnicamente. Está indicando que ese atributo no debe usarse externamente.
- **Doble guion inicial (__atributo)**: Activa el **Name Mangling** (Modificación de Nombres). Cambia el nombre internamente a **_Clase__atributo** esto evita que este atributo sea sobrescrito por una subclase, además daría error si externamente intentamos hacer **print(Clase.__atributo)**.

- **Doble guion inicial y final (__init__)**: métodos especiales del lenguaje, sin name mangling. También se conocen como **dunder methods** (double underscore).

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self._activo = False
        self.__puerto = puerto
    def iniciar(self):
        self._activo = True
    def get_activo(self):
        return self._activo
    def set_activo(self, activo: bool):
        self._activo = activo
    def get_puerto(self):
        return self.__puerto
    def set_puerto(self, puerto: str):
        self.__puerto = puerto

SSH=Servicio("ssh",22)
print(SSH.nombre) # Muestra ssh
print(SSH.get_activo()) # Muestra False
print(SSH.get_puerto()) # Muestra 22
print(SSH._activo) # Muestra False
print(SSH.__puerto) # AttributeError
print(SSH._Servicio__puerto) # Muestra 22
```

Propiedades con @property y setter

En Python moderno no es habitual crear métodos **getX()** y **setX()** para todo. Si necesitamos controlar acceso o validación, se pueden usar propiedades con **@property**.

- Podemos pensar en una propiedad como un método que se hace pasar por un atributo.
- La propiedad se lee como un atributo normal, pero internamente ejecuta un método. Esto mantiene una interfaz limpia.
- **setter** se utiliza para hacer la asignación de la propiedad y permite validar o corregir el valor antes de aceptar un cambio. Por ejemplo, convertir a minúsculas y eliminar espacios.

```
class Servicio:
    def __init__(self, nombre):
        # Al no poner _nombre usa el setter
        self.nombre = nombre

    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, valor):
        # setter usa _nombre y no nombre
        # Esto evita un bucle infinito
        # de asignacion
        self._nombre = valor.strip().lower()

SSH=Servicio(" Ssh ")
print(SSH.nombre)
```

Validar estado dentro de la clase

Una clase puede proteger sus invariantes: reglas que deben cumplirse para que el objeto esté en un estado válido.

- Por ejemplo, un puerto debe estar entre 1 y 65535. Si cada parte del programa modifica el atributo directamente, la validación se pierde.
- Centralizar la validación dentro de la clase reduce errores y hace el código más mantenible.
- **NOTA: raise** eleva un error con un mensaje más amigable que posteriormente puede ser tratado

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    @property
    def puerto(self):
        return self._puerto

    @puerto.setter
    def puerto(self, valor):
        if not 1 <= valor <= 65535:
            raise ValueError("Puerto no válido")
        self._puerto = valor
```

Composición

La composición consiste en construir una clase usando objetos de otras clases como atributos.

- En lugar de heredar, una clase contiene otra.
- Es una técnica muy útil para dividir problemas: una alerta puede tener un destino, un servicio puede tener una configuración o un servidor puede tener una interfaz de red.
- **Regla práctica:** si la relación es “tiene un”, probablemente conviene composición.
- La composición suele producir diseños más flexibles que una jerarquía de herencia prematura.
- No hay que forzar herencia cuando la relación natural es contener o usar otro objeto.

En el ejemplo, Servidor no hereda de **InterfazRed**. Simplemente tiene una interfaz asociada. Esa separación permite cambiar la interfaz, probarla por separado o reutilizarla en otra clase.

```
class InterfazRed:
    def __init__(self, nombre, ip):
        self.nombre = nombre
        self.ip = ip

class Servidor:
    def __init__(self, hostname, interfaz):
        self.hostname = hostname
        self.interfaz = interfaz

eth0 = InterfazRed("eth0", "10.0.0.20")
srv = Servidor("srv-web-01", eth0)

print(f"{srv.hostname} -> {srv.interfaz.ip}")

# Cambiar la interfaz
srv.interfaz = InterfazRed("eth1", "10.0.1.20")
print(srv.interfaz.nombre)
```

Herencia: especializar una clase

La herencia permite **crear una clase nueva a partir de otra**. La clase hija hereda atributos y métodos de la clase base y puede añadir o modificar comportamiento.

- Tiene sentido cuando la relación es “es un tipo de”. Por ejemplo, un ServicioWeb puede ser un tipo especializado de Servicio.
- No debe usarse solo para compartir código: muchas veces la composición explicada antes es más clara.

```
class Servicio:
    def __init__(self, nombre):
        self.nombre = nombre

    def estado(self):
        return f"{self.nombre}: definido"

class ServicioWeb(Servicio):
    pass

print(ServicioWeb("nginx").estado())
```

super() y reutilización del inicializador

Cuando una clase hija necesita inicializar la parte heredada, se usa **super().__init__()**. Así se reutiliza el código de la clase base y se evita duplicación.

- Después, la clase hija puede añadir sus propios atributos.
- Este patrón es habitual en herencia simple y facilita mantener el código cuando cambia la clase base.

```
class Servicio:
    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

class ServicioWeb(Servicio):
    def __init__(self, nombre, puerto, activo=False):
        super().__init__(nombre, activo)
        self.puerto = puerto

web1 = ServicioWeb("nginx", 80, True)
web2 = ServicioWeb("http", 8080)

print(f"Servicio Web1: {web1.nombre} -> {web1.puerto}:{web1.activo}")
# Servicio Web1: nginx -> 80:True
print(f"Servicio Web2: {web2.nombre} -> {web2.puerto}:{web2.activo}")
# Servicio Web2: http -> 8080:False
```

Sobrescritura de métodos

Una clase hija puede redefinir un método heredado. A esto se le llama sobrescritura. Permite adaptar el comportamiento sin cambiar la clase base.

- Si se necesita conservar parte del comportamiento original, se puede llamar al método de la clase base mediante **super()**.
- Conviene sobrescribir solo cuando la especialización aporta claridad.

```
class Servicio:
    def descripcion(self):
        return "Servicio genérico"

class ServicioWeb(Servicio):
    def descripcion(self):
        return "Servicio web HTTP/HTTPS"

servicio1 = Servicio()
servicio2 = ServicioWeb()
print(servicio1.descripcion())
# Servicio genérico
print(servicio2.descripcion())
# Servicio web HTTP/HTTPS
```

Polimorfismo básico

El polimorfismo permite tratar objetos distintos de diferentes clases mediante una misma operación si todos ofrecen el método esperado con el mismo nombre.

No es necesario preguntar siempre por el tipo concreto del objeto. Si tiene el método que necesitamos, podemos usarlo.

En Python esto favorece código flexible, pero exige nombres de métodos coherentes y comportamiento previsible.

```
class ServicioWeb:
    def comprobar(self):
        return "HTTP OK"

class ServicioBD:
    def comprobar(self):
        return "BD OK"

for servicio in [ServicioWeb(), ServicioBD()]:
    print(servicio.comprobar())
```

Herencia múltiple

Python permite que una clase herede de varias clases.

- Esta posibilidad existe, pero debe usarse con cautela porque puede complicar el orden de resolución de métodos y el diseño.
- Si una clase solo necesita usar otra funcionalidad, probablemente la composición sea más sencilla.

```
class Monitorizable:
    def comprobar(self):
        return "comprobado"

class Reinicialable:
    def reiniciar(self):
        return "reiniciado"

class ServicioSistema(Monitorizable, Reinicialable):
    pass

svc = ServicioSistema()
print(svc.comprobar(), svc.reiniciar())
```

Composición frente a herencia

- La herencia expresa “es un tipo de”.
- La composición expresa “tiene un” o “usa un”.
- Elegir mal puede crear clases difíciles de extender.
- Como criterio general:
 - empieza con composición cuando solo necesites combinar piezas.
 - Usa herencia cuando exista una especialización clara y estable.
 - Un diseño sencillo suele ser más valioso que una jerarquía grande.

```
class Servicio:
    pass

# Herencia: ServicioWeb es un tipo de Servicio
class ServicioWeb(Servicio):
    pass

# Composición: Servidor tiene servicios
class Servidor:
    def __init__(self, servicios):
        self.servicios = servicios
```

Clases en scripts de TI

No todos los scripts necesitan clases.

- Para tareas pequeñas, la programación estructurada con funciones y datos simples pueden ser suficiente.
- Las clases empiezan a aportar valor cuando hay varios datos relacionados, operaciones asociadas y necesidad de mantener estados.

Por ejemplo: representar servidores, usuarios, reglas, tareas, métricas, alertas o resultados de comprobaciones.

```
class ResultadoChequeo:
    def __init__(self, nombre, ok, detalle):
        self.nombre = nombre
        self.ok = ok
        self.detalle = detalle

    def resumen(self):
        estado = "OK" if self.ok else "FALLO"
        return f"{self.nombre}: {estado} - {self.detalle}"
```

Buenas prácticas y Errores Frecuentes

- **Buenas Prácticas:**
 - Usar nombres de clase en **PascalCase** y nombres de atributos/métodos en **snake_case**.
 - Mantener clases con una responsabilidad clara y evitar objetos que hagan de todo.
 - Inicializar los atributos importantes en **__init__()** para que el objeto empiece en un estado previsible.
 - Preferir **f-strings** para construir mensajes y **__str__()** para una salida textual legible.

- Documentar la intención cuando la clase no sea evidente.
- Olvidar self en la definición de un método de instancia.
- Crear atributos fuera de `__init__()` sin una razón clara.
- **Errores Frecuentes:**
 - Modificar atributos internos sin pasar por una validación necesaria.
 - Usar herencia cuando bastaba con una composición.
 - Confundir clase con objeto:
 - La clase define la plantilla.
 - El objeto (la instancia) contiene los valores concretos.
 - También es frecuente imprimir dentro del método cuando debería devolverse una cadena reutilizable.

```
class TareaProgramada:
    def __init__(self, nombre, hora):
        self.nombre = nombre
        self.hora = hora

    def __str__(self):
        return f"{self.nombre} a las {self.hora}"

print(TareaProgramada("backup", "02:00"))
```

Resumen del Tema 4

- Una clase define atributos y métodos.
- Un objeto es una instancia concreta de una clase.
- `__init__()` inicializa el estado y `self` permite acceder a los datos de cada instancia.
- La encapsulación ayuda a controlar cambios de estado.
- La composición permite crear objetos a partir de otros objetos.
- La herencia permite especializar una clase base, pero debe usarse con criterio.

El siguiente tema profundizará en colecciones de objetos: listas, tuplas, diccionarios, conjuntos y operaciones sobre grupos de datos.

```
class Concepto:
    def resumen(self):
        return "Clases listas para modelar objetos"

print(Concepto().resumen())
# Siguiendo paso: colecciones de objetos
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema04.

El laboratorio debe reforzar la diferencia entre clase e instancia, el uso de `self`, la representación con `__str__()`, la composición y la herencia simple.

- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python --version
# Actualizar del repositorio Git
git pull origin main
# Para ejecutar VSCode
code . # usar carpeta Tema04
# Para ejecutar en Terminal
python
```